Proposta:

Desenvolver uma aplicação web em React com TypeScript, utilizando Redux Toolkit para gerenciamento de estado global. A aplicação deverá consumir a API pública de previsão do tempo do INPE (CPTEC), permitindo que o usuário digite o nome de uma cidade (com recurso de autocomplete) e visualize a previsão dos próximos 4 dias.

O sistema terá:

- Um campo de busca para o nome da cidade.
- Sugestões automáticas enquanto o usuário digita (autocomplete).
- Botão para buscar previsão.
- Exibição das condições climáticas (mínima, máxima, tempo previsto) para 4 dias.

Objetivos:

1. Praticar conceitos de React com TypeScript, estruturando componentes funcionais tipados.
2. Aplicar Redux Toolkit para centralizar o estado da aplicação e lidar com requisições assíncronas via createAsyncThunk.
3. Consumir APIs externas (INPE/CPTEC) utilizando fetch e tratamento de dados no formato XML.
4. Implementar autocomplete para busca de cidades, trabalhando com listas e eventos controlados.
5. Organizar o projeto com Vite, configurando a estrutura inicial e preparando o ambiente de desenvolvimento.
6. Trabalhar boas práticas de código (separação por pastas, comentários explicativos, tipagem forte, uso de slices no Redux).

Será criado uma pasta chamada **previsão-tempo**

I. Preparação do ambiente de desenvolvimento

Passo 1 - Criar um projeto Vite com template React + TypeScript:

Para reproduzir os exemplos, crie um projeto React TS usando a ferramenta de build Vite:

```
npm create vite@latest previsao-tempo -- --template react-ts
```

Escolha o freamework: **React** e a variant: **TypeScript**

```
cd previsao-tempo
npm install
```

Passo 2 - Instalar as dependências do Redux

```
npm install @reduxjs/toolkit react-redux
```

```
npm install --save-dev @types/xml2js
```

Tipo de dependência: devDependencies (--save-dev) → usada apenas durante o desenvolvimento, não é necessária em produção. Fornece **tipagens TypeScript** para a biblioteca xml2js. O TypeScript precisa saber os tipos dos objetos e funções que você está usando. Com @types/xml2js, você consegue:

- Evitar erros de tipagem
- Ter **autocompletar** no editor
- Trabalhar com segurança com XML no TypeScript

```
npm install fast-xml-parser
```

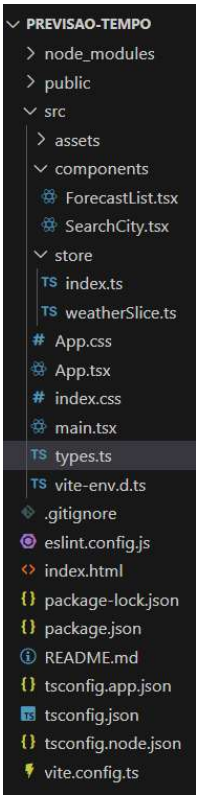
Tipo de dependência: dependencies (padrão) → usada em produção e desenvolvimento. Ele converte XML em JSON e JSON em XML rapidamente. No projeto ele**:**

- recebe os dados do **INPE em XML**
- fast-xml-parser transforma esse XML em um objeto JavaScript/TypeScript
- Facilita acessar propriedades como cidadeData.cidades.cidade

Vantagens:

- Mais rápido que xml2js
- Não precisa de callbacks (suporta Promises)
- Suporta várias opções de parsing, como ignorar atributos ou valores nulos

Passo 3 – Criar a estrutura do projeto



Passo 4 – Criando os tipos da previsão `src/types.ts`

```ts
// Estrutura da previsão que vamos usar no Redux
export interface Forecast {
  dia: string;        // Data da previsão
  tempo: string;      // Código do clima (ex: "pn" = parcialmente nublado)
  maxima: number;     // Temperatura máxima
  minima: number;     // Temperatura mínima
}
```

Passo 5 – Criando o Redux Slice - `src/store/weatherSlice.ts`

```ts
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import type { Forecast } from "../types";
import { XMLParser } from "fast-xml-parser";

// ------------------------------
// Função para remover acentos e cedilha
// Ex.: "São Paulo" → "Sao Paulo"
// ------------------------------
function sanitizeCityName(city: string): string {
  return city
    .normalize("NFD") // separa letra e acento
    .replace(/[\u0300-\u036f]/g, "") // remove acentos
    .replace(/ç/g, "c") // troca cedilha
    .trim(); // remove espaços extras no início/fim
}

// ------------------------------
// Definição do estado do Redux
// ------------------------------
interface WeatherState {
  forecasts: Forecast[]; // Lista de previsões (array de objetos Forecast)
  status: "idle" | "loading" | "succeeded" | "failed"; // Estado da requisição
  error: string | null; // Mensagem de erro (se houver)
}
```

```
// ------------------------------
// Thunk assíncrono para buscar previsão do INPE
// ------------------------------
export const fetchWeather = createAsyncThunk(
  "weather/fetchWeather", // Nome da ação
  async (city: string) => {
    const parser = new XMLParser(); // Parser XML → JSON

    // Sanitizar o nome da cidade
    const sanitizedCity = sanitizeCityName(city);

    // Buscar o código da cidade no INPE
    const cityResponse = await fetch(
      `http://servicos.cptec.inpe.br/XML/listaCidades?city=${sanitizedCity}`
    );
    const cityText = await cityResponse.text();
    const cityData: any = parser.parse(cityText); // Converter XML → JSON

    // Tratar caso a cidade não seja encontrada
    if (!cityData.cidades?.cidade) {
      throw new Error("Cidade não encontrada."); // Esse erro será capturado pelo Redux
    }
```

```
  // O INPE pode devolver 1 cidade ou uma lista
  const cityCode = Array.isArray(cityData.cidades.cidade)
    ? cityData.cidades.cidade[0].id // pegar primeira cidade da lista
    : cityData.cidades.cidade.id;   // única cidade

  // Buscar a previsão de 7 dias da cidade (vamos usar só 4)
  const forecastResponse = await fetch(
    `http://servicos.cptec.inpe.br/XML/cidade/7dias/${cityCode}/previsao.xml`
  );
  const forecastText = await forecastResponse.text();
  const forecastData: any = parser.parse(forecastText);

  // Transformar os dados do INPE no formato do nosso tipo Forecast
  const forecasts: Forecast[] = forecastData.cidade.previsao
    .slice(0, 4) // Pegamos apenas os 4 primeiros dias
    .map((item: any) => ({
      dia: item.dia,             // Data da previsão
      tempo: item.tempo,         // Código do tempo (ex: "pn" = parcialmente nublado)
      maxima: Number(item.maxima), // Temperatura máxima
      minima: Number(item.minima), // Temperatura mínima
    }));
```

```
    // Retornar para o Redux
    return forecasts;
  }
);

// ------------------------------
// Criando o Slice do Redux
// ------------------------------
const weatherSlice = createSlice({
  name: "weather",
  initialState,
  reducers: {}, // Não temos reducers manuais, só via Thunk
  extraReducers: (builder) => {
    builder
      // Quando a requisição começa
      .addCase(fetchWeather.pending, (state) => {
        state.status = "loading";
        state.error = null; // limpar erro anterior
      })
      // Quando a requisição foi concluída com sucesso
      .addCase(fetchWeather.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.forecasts = action.payload; // Previsões recebidas
      })
```

```
      // Quando ocorreu algum erro
      .addCase(fetchWeather.rejected, (state, action) => {
        state.status = "failed";
        state.error = action.error.message || "Erro ao buscar previsão.";
      });
  },
});

export default weatherSlice.reducer;
```

1. createAsyncThunk
   - Serve para lidar com chamadas assíncronas (API).
   - Aqui usamos para buscar a previsão no INPE.

2. fast-xml-parser
   - Converte XML → JSON de forma simples.
   - Assim, podemos manipular os dados como objetos no React.

3. Estrutura do Estado (WeatherState)
   - forecasts: lista de previsões.
   - status: controla se está carregando, teve sucesso ou erro.

- error: guarda a mensagem de erro, caso falhe.

4. extraReducers
   - Captura os três estados possíveis da requisição:
     - pending → carregando
     - fulfilled → sucesso
     - rejected → erro

Passo 6 – Configurar o Redux através do arquivo `src/store/index.ts`

A Store é o local onde o estado global da aplicação será armazenado. Configura a store do Redux, que é o "cérebro" onde todo o estado global fica. Geralmente importa o configureStore do @reduxjs/toolkit, define quais *reducers* serão usados e exporta o tipo RootState e AppDispatch. É o ponto central para registrar todos os estados globais da aplicação.

```
import { configureStore } from "@reduxjs/toolkit";
import weatherReducer from "./weatherSlice";

export const store = configureStore({
  reducer: {
    weather: weatherReducer,
  },
});

// Tipos para usar no React com segurança
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Passo 7 – Configurar o buscar cidade através do arquivo `src/components/SearchCity.tsx`

```
import { useState } from "react";
import { useDispatch } from "react-redux";
import type { AppDispatch } from "../store";
import { fetchWeather } from "../store/weatherSlice";

export default function SearchCity() {
  const [city, setCity] = useState("");
  const dispatch = useDispatch<AppDispatch>();

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (city.trim() !== "") {
      dispatch(fetchWeather(city));
    }
  };
```

```
  return (
    <form onSubmit={handleSubmit} className="p-4">
      <input
        type="text"
        placeholder="Digite o nome da cidade"
        value={city}
        onChange={(e) => setCity(e.target.value)}
        className="border p-2 rounded"
      />
      <button
        type="submit"
        className="ml-2 bg-blue-500 text-white px-4 py-2 rounded"
      >
        Buscar
      </button>
    </form>
  );
}
```

Passo 8 – Configurar o listar cidade através do arquivo `src/components/ ForecastList.tsx`

```
import { useSelector } from "react-redux";
import type { RootState } from "../store";

export default function ForecastList() {
  const { forecasts, status, error } = useSelector(
    (state: RootState) => state.weather
  );

  if (status === "loading") return <p>Carregando...</p>;
  if (status === "failed") return <p>Erro: {error}</p>;
  if (forecasts.length === 0) return <p>Nenhuma previsão encontrada.</p>;

  return (
    <div className="p-4">
      <h2 className="text-lg font-bold mb-2">Previsão para 4 dias:</h2>
      <ul>
        {forecasts.map((f, i) => (
          <li
            key={i}
            className="border rounded p-2 mb-2 shadow bg-gray-100"
          >
            <p><strong>Data:</strong> {f.dia}</p>
            <p><strong>Clima:</strong> {f.tempo}</p>
            <p><strong>Máx:</strong> {f.maxima}°C | <strong>Mín:</strong> {f.minima}°C</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Passo 9 – Configurar o App através do arquivo `src/App.tsx`

```
import SearchCity from "./components/SearchCity";
import ForecastList from "./components/ForecastList";

export default function App() {
  return (
    <div className="max-w-lg mx-auto">
      <h1 className="text-2xl font-bold text-center my-4">
        Previsão do Tempo (INPE)
      </h1>
      <SearchCity />
      <ForecastList />
    </div>
  );
}
```

Passo 10 – Configurar o `src/main.tsx`

```
src > main.tsx
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { Provider } from "react-redux";
import { store } from "./store";

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```

Passo 11 – Executando o projeto

PS C:\previsao-tempo> npm run dev

> previsao-tempo@0.0.0 dev

> vite

VITE v7.1.4 ready in 273 ms

- ➜ **Local:** **http://localhost:5173/**
- ➜ **Network: use --host to expose**
- ➜ **press h + enter to show help**